

формальная спецификация на игру «Операция Озверин:
принуждение ребят к дружной жизни» (далее «Ozver in»)

Преамбула

Настоящий документ является подведением итогов теоретического исследования и концептуализации проекта «Ozverin» и содержит в себе теоретическую базу вычислительного ядра. Изложены составляющие системы и её инварианты с использованием формализма TLA+. Тем не менее, настоящая формальная спецификация является лишь ориентировочным документом, а не точным изложением на языке TLA+. Её суть заключается в переводе интуитивных догадок и философских посылок на строгий язык формальной логики и математики.

Коротко о проекте

Проект «Ozverin» – это инициатива пошаговой стратегической игры, основанной на теории автоматов, теории графов и движимой данными (Data-Driven) архитектуры. Смысл игры заключается в создании максимально реалистичной, математически обоснованной военно-политической симуляции с элементами ролевого отыгрыша и нарративного повествования, предоставляющих широкое пространство для творческой свободы игроков.

Основные компоненты технического проекта

Настоящий проект включает в себя вычислительное ядро, состоящего из двусвязного циклического ориентированного графа, представляющего собой систему сущностей; конечных автоматов Мили, каждый из которых отражает конкретную сущность, тип связи (горизонтальная, вертикальная, прямая и обратная) и правила взаимодействия (триггерные схемы); данных, определяющих институциональную структуру графа, агентскую архитектуру NPS, ролевые компоненты, характеристики объектов и прочее.

Помимо вычислительного ядра, проект также включает интерфейс, интерактивную воксельную карту, трёхуровневую систему сохранений, графический редактор контента и некоторые мелкие дополнения, в настоящей формальной спецификации не затрагиваемые.

Данная формальная спецификация излагает только поведение системы и составные элементы вычислительного ядра. В свете этого, интерфейсные характеристики, асимметричный дизайн и всё другое, не являющееся поведением, в данной спецификации не рассматривается.

Аксиомы и теоретические посылки

Минимальный класс игровых сущностей – это закрытая термодинамическая системная машина, выраженная через конечный автомат Мили.

Её свойства:

1. Способность принимать что-то на вход (Input);
2. Способность преобразовывать входные данные (Process);
3. Способность выводить выходные данные (Output);
4. Внутреннее состояние, определяющие три вышеизложенные способности, выраженное в битовой маске (State);
5. Стадия развития, включённая во внутреннее состояние (S-образная кривая – младенчество, детство, юность, зрелость, старость, смерть или метаморфоза);
6. Системный эффект, не допускающий редукцию до суммы отдельных компонентов;
7. Дуализм потребностей, заключающийся в стремлении, с одной стороны, к выживанию, а с другой стороны, к расширению своего влияния (9);
8. Самоподобие, означающее возможность суждения о целом по его части (8);
9. Существование в конкретной среде;
10. Самостоятельное изменение своего состояния посредством собственных калькуляторов и триггерных схем на основании входящих сигналов;
11. Разделение буферов чтения состояния на текущем тике и записи на следующий тик;
12. Наличие порогового механизма с гистерезисом и учётом устойчивого, но недостаточно сильного давления.

Минимальный класс мира – это граф, представленный в виде сущностей, как вершин.

Типографские соглашения

En – сущность;

En_p – родительская сущность;

En_c – дочерняя сущность;

t – тик;

σ – параметр проводимости;

Wl – мир.

Формальные определения

$$En = (Q, \Sigma, \Lambda, \delta, \lambda, q_0) \quad (1)$$

Q – конечное множество состояний.

Σ – входной алфавит.

Λ – выходной алфавит.

$\delta: Q * \Sigma \rightarrow Q$ – функция перехода.

$\lambda: Q * \Sigma \rightarrow \Lambda$ – функция выхода.

$q_0 \in Q$ – начальное состояние.

$$q \in Q = (b_1, b_2, \dots, b_{64}), \forall b_i \in \{0, 1\}$$

$$\Sigma = (En_p^\lambda + En_c^\delta)$$

$$\forall \sigma_{uv} \in [0, 1]$$

u, v – соседние вершины графа (связанные общим контуром En).

$$q'_i = \begin{cases} 1, \text{ если } p_i > \theta_{high} \\ 0, \text{ если } p_i < \theta_{low} \\ q_i, \text{ если } \theta_{low} \leq p_i \leq \theta_{high} \end{cases} \quad (2)$$

p_i – взвешенная доля откликов дочерних вершин.

$$p_i = \sum_{j=1}^J \frac{n_j}{\sum_{j=1}^J n_j} * \sigma_{u_j v} * (\frac{1}{n_j} \sum_{r \in R_j} r[i])$$

R_j – множество откликов произвольной дочерней вершины;

r_i – значение i -го бита в отклике r ;

n_j – количество откликов произвольной вершины за такт, каждый из которых является Λ .

$$\theta_{low} = \frac{1}{2} - \frac{k}{\sqrt{N_{total}}}$$

$$\theta_{high} = \frac{1}{2} + \frac{k}{\sqrt{N_{total}}}$$

$k \in [0.5, 2.0]$ – коэффициент чувствительности.

$$U_i^{t+1} = \alpha \cdot U_i^t + (p_i - q_i^t)$$

U_i^{t+1} – коэффициент накопленного напряжения в следующем тике для произвольного бита;

$\alpha \in [0, 1)$ – коэффициент затухания напряжения;

$p_i - q_i^t$ – мгновенное рассогласование между сигналом дочерних вершин и текущим состоянием родительской вершины.

$$q_i^{t+1} = \begin{cases} 1, & \text{если } U_i^{t+1} > \frac{1}{1-\alpha} \\ 0, & \text{если } U_i^{t+1} < -\frac{1}{1-\alpha} \\ q_i^t & \text{в противном случае} \end{cases}$$

$U_i^{t+1} = 0$ при выполнении условий переключения.

$$I_k = e_k \cdot s_k \quad (4)$$

$e_k \in [0, 1]$ – сила воздействия модальности k вектора модальностей E , описывающего генерируемые средой события;

$s_k \in [0, 1]$ – мера чувствительности сущности к k -ой модальности профиля чувствительности S .

$$M_{k,i} = (s_{k,i}, \tau_{inst}, \theta_{acc}, \beta_{k,i}) \quad (3)$$

M – матрица влияния;

τ_{inst} – мгновенный порог переключения;

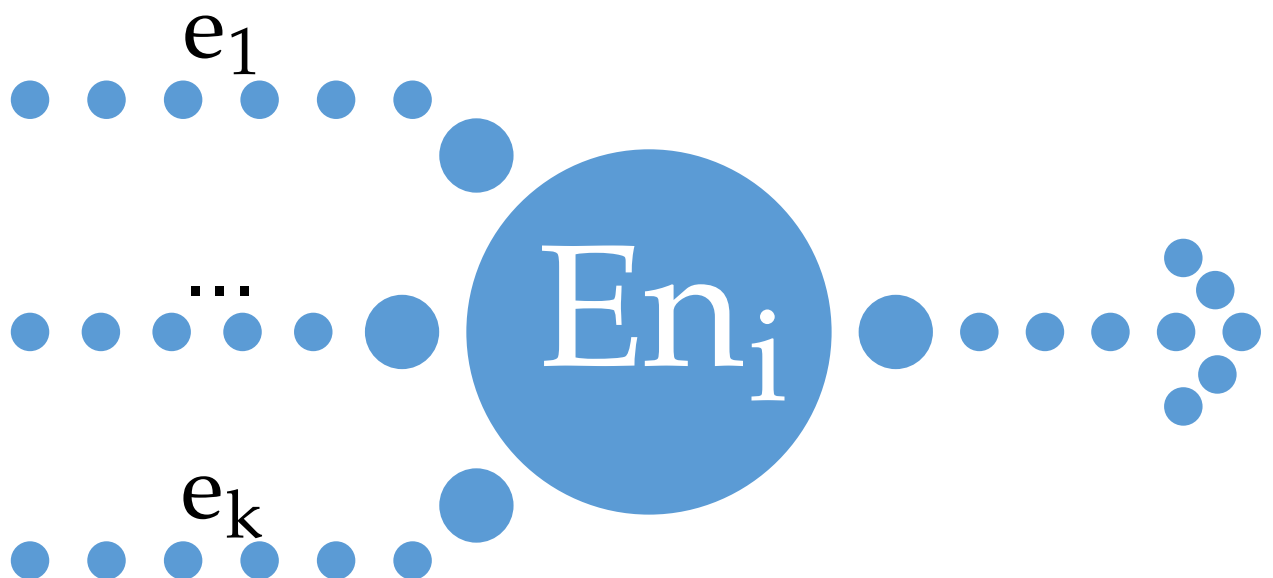
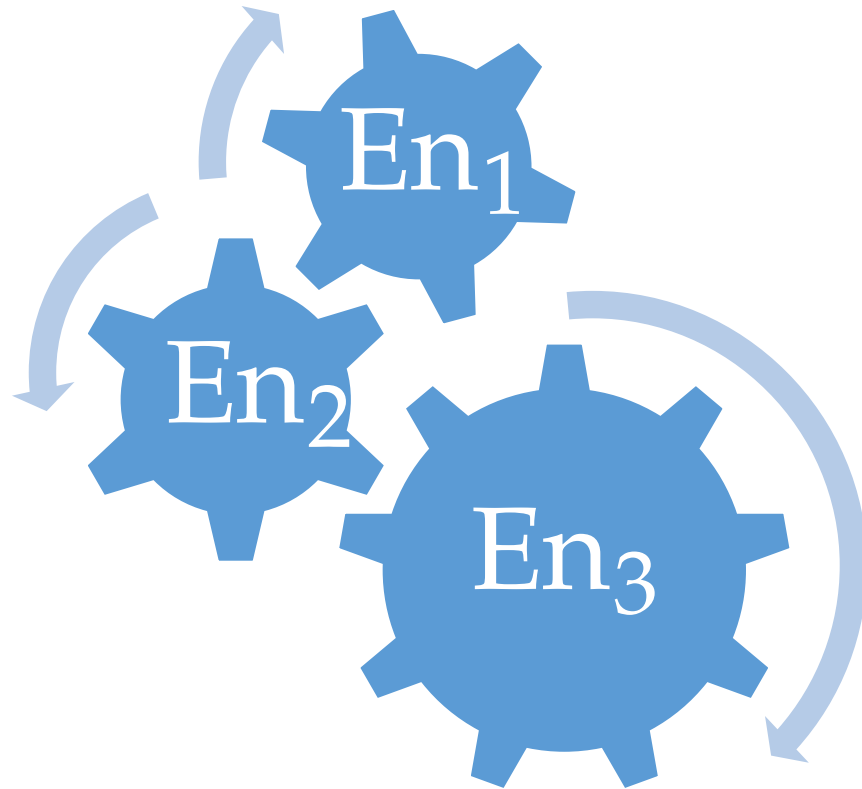
θ_{acc} – порог накопления напряжения;

$\beta \in [0, 1)$ – коэффициент затухания.

$$U_{env,i}^{t+1} = \beta \cdot U_{env,i}^t + \sum_k I_k \cdot [k \rightarrow i]$$

$$K_{eff} = K_{base} \cdot m(S, U)$$

Архитектура



Формальные декларации переменных и констант

VARIABLES BitMask, VoltageAccumulation, ModalitiesVector, EntitiesQuantity, CurrentTick, LodStatus, BdiState, ScmittThresholdLow, ScmittThresholdHigh, CoefficientEffective, ObjectiveFunctionWeights, InfluenceMatrix, GraphTopology, Compositions, SusceptibilityProfile, Chronicle, Event, Entity, segment_name, value, threshold, voltage_name, delta, flag, state, connection, signal, event.

BitMask – значение битовой маски состояния сущности;

VoltageAccumulation – мера накопления напряжения при устойчивом, но недостаточном для преодоления порога Шмитта давлении;

ModalitiesVector – эффективный вектор модальностей среды в конкретное время в конкретном месте;

EntitiesQuantity – количество сущностей в графе (мире);

CurrentTick – текущий тик по концепции OODA (Observe, Orient, Decide, Act);

LodStatus – мера удалённости сущности от игрока, определяющая периодичность обновления характеристик;

BdiState – состояние интеллектуального агента по модели BDI (Belief, Desire, intention) (7);

ScmittThresholdLow и ScmittThresholdHigh – эффективные пороги Шмитта;

CoefficientEffective – эффективные коэффициенты формул;

ObjectiveFunctionWeights – эффективные веса целевой функции;

InfluenceMatrix – матрица влияния;

GraphTopology – топология графа;

Compositions – логические композиты, производные от предикатов (6);

SusceptibilityProfile – эффективный профиль восприимчивости сущностью модальностей среды (4);

Chronicle – нарративная история игровых событий;

Event – игровое событие;

Entity – сущность (1).

CONSTANTS BitMaskLenght, Domains, IpoFunctions, TactsPerPhase, LodLeves, BitsPerPredicate, SusceptibilityProfileBase, ScmittThresholdLowBase, ScmittThresholdHighBase, ObjectiveFunctionWeightsBase, ValidationRules, CompositionRules, MetamorphosisRules, GlobalRules, ModalitiesVectorGlobal, eid, input_alphabet, output_alphabet, config.

BitMaskLenght – длина битовой маски состояния сущности: 64 бита;

Domains – количество доменов, 4: материя, энергия, информация, сила;

IpoFunctions – количество IPO-функций;

TactsPerPhase – число тактов на фазу, 3: чтение из текущего буфера, запись в следующий буфер, вычисление и обновление;

LodLevels – число уровней LOD (levels of detalization), определяемое на этапе компиляции, 3: полная симуляция, эвристическая модель, статистическая прогрессия; (5)

BitsPerPredicate – число битов, описывающих один предикат (или число предикатов, описываемых n количеством битов), определяет схему маски; (6)

SusceptibilityProfileBase – базовая восприимчивость среды; (3)

ScmittThresholdLowBase и ScmittThresholdHighBase – базовые значения нижнего и верхнего порогов Шмитта; (2)

ObjectiveFunctionWeightsBase – базовые веса целевой функции;

ValidationRules – правила валидации комбинации значений битов в маске, «законы социальной природы»;

CompositionRules – правила вычисления композитов; (6)

MetamorphosisRules – правила метаморфозы;

GlobalRules – мировые правила (законы природы);

ModalitiesVectorGlobal – вектор всех возможных модальностей среды.

Типы данных и интерфейсы

$\text{Entity} \triangleq [\text{eid}: \text{EntityId}, \text{state}: \text{EntityState}, \text{input_alphabet}: \text{EntityInputAlphabet}, \text{output_alphabet}: \text{EntityOutputAlphabet}, \text{config}: \text{EntityConfig}]$.

$\text{EntityId} \triangleq [0: |\text{EntitySet}|, \text{nextId}, \text{id}]$.

$\text{EntityState} \triangleq 2^{64}$.

$\text{EntityConfig} \triangleq [\text{ValidationRules}, \text{CompositeRules}, \text{MetamorphosisRules}, \text{BehaviorRules}, \text{ConductivityRules}]$.

$\text{Conditions} \triangleq [\text{BitEquals}, \text{BitGreater}, \text{VoltageExceeds}, \text{EntityExists}, \text{HasFlag}]$.

$\text{BitEquals}(\text{segment_name}, \text{value}) \triangleq \text{IF } \text{segment_name} == \text{value} \text{ THEN TRUE ELSE FALSE}$.

$\text{BitGreater}(\text{segment_name}, \text{threshold}) \triangleq \text{IF } \text{segment_name} > \text{threshold} \text{ THEN TRUE ELSE FALSE}$.

$\text{VoltageExceeds}(\text{voltage_name}, \text{threshold}) \triangleq \text{IF } \text{voltage_name} > \text{threshold} \text{ THEN TRUE ELSE FALSE}$.

$\text{EntityExists}(\text{eid}) \triangleq \text{IF } \exists \text{ eid} \text{ THEN TRUE ELSE FALSE}$.

$\text{HasFlag}(\text{flag}) \triangleq \text{IF } \exists \text{ flag} \text{ THEN TRUE ELSE FALSE}$.

$\text{Actions} \triangleq [\text{SetBit}, \text{AddVoltage}, \text{SendSignal}, \text{SpawnEntity}, \text{DestroyEntity}, \text{LogEvent}]$.

$\text{Setbit}(\text{segment_name}, \text{value}) \triangleq \text{segment_name} = \text{value}$.

$\text{AddVoltage}(\text{voltage_name}, \text{delta}) \triangleq \text{voltage_name} + \text{delta}$.

$\text{SendSignal}(\text{target_id}, \text{signal_dict}) \triangleq \text{Send}(\text{target_id}, \text{signal_dict})$.

$\text{SpawnEntity}(\text{class_name}, \text{config}) \triangleq 0 + \text{Entity}(\text{config}=(\text{class_name}, \text{config}))$.

$\text{DestroyEntity}(\text{eid}) \triangleq 0 - \text{Entity}(\text{eid}=\text{eid}).$

$\text{LogEvent}(\text{Event}) \triangleq \text{Chronicle} + \text{Event}.$

ASSUME $\text{ValidationRules} \neq \text{CompositeRules},$

$\wedge \text{ValidationRules} \neq \text{MetamorphosisRules},$

$\wedge \text{ValidationRules} \neq \text{BehaviorRules},$

$\wedge \text{ValidationRules} \neq \text{ConductivityRules},$

$\wedge \text{CompositeRules} \neq \text{MetamorphosisRules},$

$\wedge \text{CompositeRules} \neq \text{BehaviorRules},$

$\wedge \text{CompositeRules} \neq \text{ConductivityRules},$

$\wedge \text{MetamorphosisRules} \neq \text{BehaviorRules},$

$\wedge \text{MetamorphosisRules} \neq \text{ConductivityRules},$

$\wedge \text{BehaviorRules} \neq \text{ConductivityRules}.$

ИНВАРИАНТЫ СИСТЕМЫ

$\text{EntityId} \triangleq \wedge O \subseteq \text{Nat}$

$\wedge \text{nextId} \in \text{Nat}$

$\wedge \text{id} \in [O \rightarrow \text{Nat}]$

$\wedge \forall o \in O: \text{id}[o] \geq 1$

$\wedge \forall o_1, o_2 \in O: (o_1 \neq o_2) \Rightarrow (\text{id}[o_1] \neq \text{id}[o_2])$

$\text{ASSUME } \text{EntityState} = 2^{64} \wedge \forall \text{state} \in \text{EntityState} = \{0, 1\}^{64}$

$\forall \text{Entity}, \forall t, \text{reads}(t) \in \text{buffer}[t] \wedge \text{writes}(t) \rightarrow \text{buffer}[t+1]$

$\forall \text{tick}_a < \text{tick}_b$

$\forall \text{connection} \in \text{entity.connections}, \text{connection.target} \in \text{World}$

$\forall \text{entity}, \text{entity} \notin \text{entity.connections.targets}$

$\forall \text{signal}, \text{signal.type} \in \text{World.registered_signal_types}$

$\forall \text{signal}, 0 \leq \text{signal.intensity} \leq 1$

$\forall \text{entity}, \forall t, \text{entity.mask} = \text{TRUE} \vee \text{validation_rules}$

$\forall \text{entity}, \text{если } \text{entity.metamorphosis_triggered}, \text{ то } \exists \text{child_entity} : \\ \text{child_entity.parent_id} == \text{entity.id}$

$\forall \text{child_entity}, \text{child_entity.inherited_segments} \subseteq \\ \text{parent_entity.config.inheritance_rules}$

$\forall \text{event}, \text{event.modality} \in \{0, 1, \dots, K-1\} \wedge K = 32$

$\forall \text{event}, 0 \leq \text{event.intensity} \leq 1$

$\forall \text{entity}, \text{IF } \text{entity.LOD} \leq 2, \text{ THEN } \text{entity.voltages} == \text{null}$

$\forall \text{entity}, \text{IF } \text{entity.LOD} < 2, \text{ THEN } \text{entity.mask} = \text{TRUE}$

Неформальное описание

Настоящий проект стратегической ролевой игры родился из чувства пресыщения текущими стратегическими играми. Я перфекционист до кончиков волос и мне дико мозолит глаза нелогичность или непоследовательность в стратегических играх. Речь идёт даже не о том, что калькуляция потерь в битвах нереалистична или чтобы убить вражеского солдата, надо ~~превратить его в решето~~ выстрелить неприемлемо много раз. Речь идёт о том, что любой желающий, если будет следовать определённой инструкции, может за относительно короткое время завоевать мир, играя за Лихтенштейн или Люксембург... И это лишь один частный, хоть и наиболее яркий пример. Меня такая клоунада не устраивает, и я решил, что напишу свою игру – с логикой, математикой и блэкджеком.

Для симуляции игровой реальности было принято решение представлять мир, как открытую, рекурсивно развивающуюся суперсистему, построенную по принципу ориентированного двунаправленного циклического графа с произвольным количеством вершин, каждая из которых спроектирована в виде конечного автомата Мили. Почему именно так? Потому что потому. Это одна из немногих аксиом проекта, невыводимая логически.

Из такой архитектуры следует, что количество возможных состояний в игре неограниченно продолженно – постоянно возникают новые вершины графа, новые варианты автоматов и тому подобное. Но даже если быть пессимистом и заявлять, что их число всё равно конечно, а уникальность вариаций растёт по логистической прямой (не линейно), такая архитектура позволяет за относительно короткий срок разработать большой, многовариативный сценарий, исчерпать возможности которого за разумный срок без автоматизации не получится. Любая новая вершина, порождающая новые связи с другими, приводит к факториальному росту возможных комбинаций связи – это простая комбинаторика. Благодаря этому обстоятельству, игрокам гарантируется свобода творческого потенциала и оригинальность игрового процесса.

Минимальный атомарный элемент суперсистемы мира, чья дальнейшая редукция без потери эмерджентных свойств невозможна (далее сущность) представлен в виде вершины графа, функционирующей по принципу конечного автомата Мили (1). Состояние каждой сущности определяется в виде 64-

битного двоичного числа (маски). Входные алфавиты формируются из сигналов от родительских вершин по контурам прямой связи и от дочерних вершин по контурам обратной связи. Выходные алфавиты направляются аналогично в обе стороны – как обратная связь родительской вершине и как прямая связь дочерней. При передаче сигналов учитываются критические пороги с гистерезисом и накопление устойчивого, но недостаточного давления (2).

Открытая система постоянно участвует во взаимодействии со средой, подразумевающее взаимное влияние. Посему, любые эффекты, исходят они от сущностей или от среды, разделены на 32 модальности:

- Физико-химические:
 - Термальные:
 - Экстремальная жара, пожар;
 - Экстремальный холод, заморозки;
 - Резкий перепад температур, термический удар;
 - Кинетические и механические:
 - Ветер, ураган, шторм;
 - Взрывная волна, ударная волна;
 - Лавина, оползень, обвал;
 - Удар, падение;
 - Гидрологические:
 - Наводнение, затопление;
 - Засуха, обезвоживание;
 - Кислотный дождь, агрессивные осадки;
 - Химические:
 - Яды, токсины;
 - Коррозия, окисление;
 - Засоление;
 - Радиационные и электромагнитные:
 - Радиация, ионизирующее излучение;
 - Электромагнитный импульс, геомагнитная буря;
 - Молния, электрический разряд.
- Биологические:
 - Патогены: вирусы, бактерии, грибки;
 - Паразиты: черви, насекомые;
 - Заживляющая среда, полезная микрофлора;

О Плодородие.

- Информационно-сенсорные:

О Ослепляющий свет, лазерное ослепление;

О Оглушающий шум, инфразвук;

О Туман, дымовая завеса;

О Сенсорная перегрузка, какофония.

Подобная архитектура делает логичными и измеримыми не только любые явления среды и влияние сущностей на среду и друг на друга, но и предоставляет широкий простор при проектировании фантастических или фэнтезийных сценариев. Пожар всегда останется пожаром и будет измеряться по тем же формулам независимо от того, устроен он огненным дыханием дракона или супер-пупер квантовым бластером. При этом, здесь важная оговорка в том, КАК эффект влияет на сущность или среду, а не ЧТО это за эффект. В этом смысле, нано-роботы, пожирающие материю для саморепликации («серая слизь» по Нейману), будут относиться к паразитам, хотя они не являются биологическими существами. Это важно учитывать для избегания логических ошибок и путаницы.

Соразмерно глобальному вектору модальностей среды, у каждой сущности есть профиль чувствительности модальностей (3). Это относится и к тем сущностям, которые по определению не взаимодействуют с некоторой модальностью – они просто имеют нулевую восприимчивость. Так, например, космический корабль невосприимчив к воде, но он в ней никогда и не находится – если находится, значит, совершенно точно его либо сбили, либо намеренно решили затопить, то есть, сущность удаляется и в дальнейших расчётах участвует только как элемент какосферы (давление на экологию).

В то же время, для оптимизации вычислений, учитывается только эффективный вектор модальностей (4), то есть, тех, которые воздействуют на конкретную сущность, в конкретном месте, в конкретное время. Вода на космический корабль в ходе фантастического боя не действует – её в космосе нет.

Также, для оптимизации и сохранения реалистичности, идёт разделение детализации проработки сущностей на три уровня (5): на первом уровне состояние сущности вычисляется каждый тик, на втором – каждые n тиков и агрегируется по предыдущим тикам, на третьем – каждые $10n$ тиков

аналогично со вторым. Уровень сущности присваивается на основании близости к ней игрока – с организационной и геометрической (географической) точек зрения.

Также, реализована система триплетов и логических композитов, позволяющих описывать сущность более полно, не меняя её битовой маски (6). То есть, комбинация двух ортогональных предикатов, тем или иным образом характеризующих сущность, вкупе с правилами вывода (определяемыми разработчиком или моддером) позволяют вывести такую характеристику сущности, которую не задавали явно, и эта характеристика будет использована в дальнейших вычислениях. Это крайне важная механика, необходимая для логичного, обоснованного процесса. Например, вождество не получит возможность стать государством до тех пор, пока не получит предикаты: аппарат принуждения (армия, дружина); система сбора прибавочного продукта (для содержания аппарата принуждения); формальные правила (чтобы аппарат принуждения не злоупотреблял и транслировал волю лидера, а не свою).

Кстати о том, как вождество станет государством. Эти события будут реализованы через механику «политических решений» – когда соблюдаются необходимые условия для прожатия решения, оно становится доступным, и при этом, не исполняется немедленно, а только начинает исполняться в следующем тике. Это усложняет и углубляет игровой процесс – не любые решения игрока сбрасывают моментально, порою он будет наткаться на сопротивление, подчас непреодолимое. Такова политика – даже диктатуры вынуждены мириться с препятствиями.

Каждый персонаж, будучи сущностью и подчиняясь тем же законам моделирования, что и танк, здание или любой иной реальный объект, является особым подвидом сущности и движим архитектурой BDI – Belief, Desire, Intention (7). Убеждения агента – это то, каким он видит окружающую обстановку. Именно убеждения, а не знания, что подразумевает возможность ложного убеждения в недостоверной информации, а это углубляет и дополняет игровой процесс, делая его более реалистичным. Желания – это потребности, их 16: голод, жажда, усталость, здоровье, страх, тревога, одиночество, репродукция, знания, творчество, скука, статус, духовность, умение, любовь, дружба. Намерения – это композиты убеждений о мире и других персонажах с потребностями. Так, комбинация

потребности голода и убеждении о том, что столовая ещё открыта и денег хватит на еду, формирует намерение сходить в столовую и что-нибудь заказать.

Любые условия выражаются в правилах: глобальных, валидации, композиции и метаморфозы. Глобальные правила – это законы природы, активные всегда. Правила валидации – это законы проверки корректности каких-либо изменений. Вода не распадётся на водород и кислород без электролиза – пример такого правила. Правила композиции – это законы комбинирования ортогональных предикатов и правила вывода из этих комбинаций. Правила метаморфозы – это законы вычисления рождения новой сущности из уже существующей, если это возможно (как например химия родилась из алхимии, или пиктограммное письмо из наскальной живописи).

Системы сущностей (дивизия, государства) выражаются в виде агрегации более низших сущностей на основании эффекта самоподобия (8). Данное обстоятельство может послужить мощным инструментом как оптимизации, так и творческого проектирования игровых сценариев.

Дуализм потребностей (9) позволяет моделировать конкуренцию между сущностями, математически строгим путём. Более подробно описано по адресу:

<https://eisenprogrammierer.github.io/Freyrsgarten/RU/%D0%9F%D1%80%D0%BE%D0%B5%D0%BA%D1%82%D1%8B/%D0%9F%D1%80%D0%BE%D0%B5%D0%BA%D1%82%20%D1%81%D1%82%D1%80%D0%B0%D1%82%D0%B5%D0%B3%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%BE%D0%B9%20%D0%B8%D0%B3%D1%80%D1%8B/%D0%9D%D0%B0%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%BA%D0%B8/%D0%A4%D0%BE%D1%80%D0%BC%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D0%B5%20%D0%BC%D0%BE%D0%B4%D0%B5%D0%BB%D0%B5%D0%B9%20%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85#%D0%B2%D0%BE%D0%BF%D1%80%D0%BE%D1%81-%D0%BA%D0%BE%D0%BD%D0%BA%D1%83%D1%80%D0%B5%D0%BD%D1%86%D0%B8%D0%B8-%D0%B8%D0%B3%D1%80%D0%BE%D0%B2%D1%8B%D1%85-%D1%81%D1%83%D1%89%D0%BD%D0%BE%D1%81%D1%82%D0%B5%D0%B8>

Спасибо за внимание!